

Linux Kernel Architecture and Programming

Embedded Linux Design and Programming

Raghav Vinjamuri

2021

Embedded Linux Design and Programming

Raghav Vinjamuri

Design Decisions to be made before development starts

The kernel is the third element of embedded Linux.

It is responsible for managing resources and interfacing with hardware, and so it affects every aspect of the final software build.

It is usually tailored to a particular hardware configuration, however
- device trees allow the creation of a generic kernel tailored to particular hardware.

how to get a kernel for a board? and
how to configure and compile it?

what is Bootstrap, focusing on the part the kernel plays?
what are device drivers and how they pick up information from the device tree?

The following main topics are the main topics discussed:

What does the kernel do?

Choosing a kernel

Building the kernel

Booting the kernel

Porting Linux to a new board

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Chapter Objectives

In This Chapter

- What does the kernel do?
- Details of Linux Architecture.
- Recap - Device Tree, bootloader
- Module Programming, procfs entry, Device Drivers
- Assignment
- Quiz

CHAPTER OVERVIEW

In This Chapter, We Will Discuss

1. What does the kernel do?
2. Details of Linux Architecture.
3. Recap - Device Tree, bootloader
4. Module Programming, procfs entry, Device Drivers
5. Assignment
6. Quiz

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Linux Architecture

Two-levels

- User Space
- Kernel-Space
- gives Linux a lot of flexibility in the way it is used

- <https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

2021

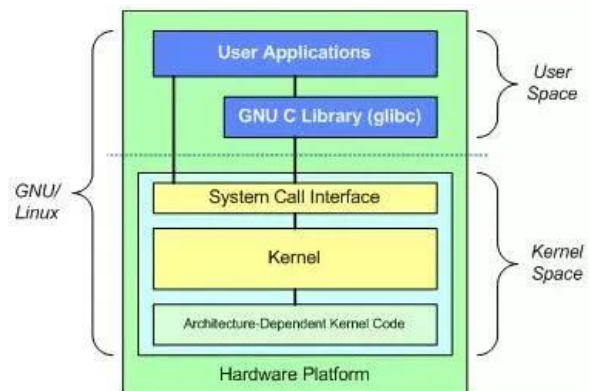
Embedded Linux Design and Programming

Raghav Vinjamuri

6-3

Linux Architecture

- is layered and based on two main levels:
 - user space, and
 - kernel space.that
 - are independent of one another and isolated.
 - communicate using system calls.
- **user space**
diagram illustrates main components.
 - **user applications, and** - first component. implement functionalities provided to embedded system user.
 - **services** - component alongside user applications. user-friendly utilities providing access to OS services.
 - **GNU C Library or glibc** - final component provides an interface between user & kernel space.



<https://www.fatalerrors.org/a/0dl20jo.html>

Linux Architecture

- **kernel space**

diagram again, illustrates main components.

- **system call interface** – first component
provides entry points to access kernel services - such as process and memory management.

- **kernel** - second component.

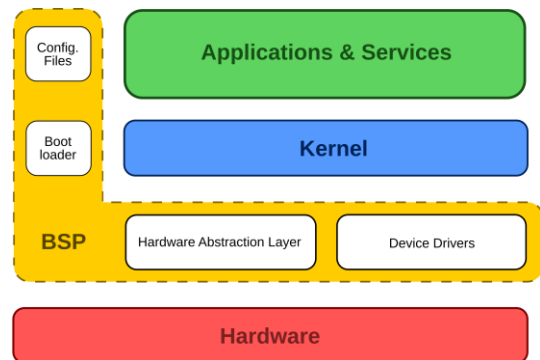
architecture-independent OS code.

implements hardware-agnostic services of OS, such as process scheduler.

- **BSP or board support package** - final component

architecture-dependent OS code.

This implements hardware-specific services of the operating system, such as context switches.



<https://www.codethink.co.uk/articles/2019/gluing-hardware-and-software-board-support-packages-bsps/>

Kernel

Conceptual View

One way, kernel is divided into five subsystems.

- process scheduler,
- memory manager,
- virtual file system,
- inter-process communication (IPC), and
- network.

- <https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Kernel – Conceptual View

One way, kernel is divided into five subsystems.

- process scheduler,
- memory manager,
- virtual file system,
- inter-process communication (IPC), and
- network.

Most of them are, in turn made up of

- hardware-independent and
- hardware-dependent

code.

Kernel

Conceptual View – Process Scheduler

process scheduler is

- responsible for decision making of what process to run at what time.
- <https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Kernel – Conceptual View

process scheduler is

- responsible for decision making of what process to run at what time.

functions in the Process Subsystem handle

- creating new copies of processes.
- CPU scheduling policy and context switch.
- interrupts and route them to appropriate kernel subsystem.
- signals to user processes.
- hardware timer.
- cleaning up process resources upon process termination.

process scheduler

- invoked from external interfaces
system calls from user space, and
kernel calls from kernel space.
- invoked on scheduler tick
called directly from system calls such as sleep, or
called indirectly after every system call or
called after every slow interrupt.

Kernel – Conceptual View

Interrupt handling types are

- **slow** is a traditional interrupt.
interrupt handling can be interrupted itself
e.g. disk driver.
- **fast** interrupt handling
interrupt handling cannot be interrupted
e.g. keyboard input..

Kernel

Conceptual View – Memory Manager

memory manager is responsible for

- large address space
- memory protection
- memory mapping
- ensure fair access to physical memory
- memory share
- using MMU
- swapping process memory out to paging file while not in use

- <https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Kernel – Conceptual View

memory manager is responsible for

- **large address space.**
user process references more than physical RAM space.
- **memory protection.**
memory for a process is private and not readable or modified by another process, unless permitted.
prevents process from overwriting text segments and read-only data.
- **memory mapping.**
map a file into virtual memory and then access file as memory
- ensure **fair access** to physical memory.
all processes have fair access to memory as a resource, and
with reasonable system performance
- **memory share.**
processes sharing a portion of their memory. e.g. code/text, shm.
- **using MMU** - memory management unit
map virtual address to physical address
- processes retain same virtual address while moving about in physical address space,
meaning, same physical memory is mapped to different processes at different times.
- **swapping process memory** out to paging file while not in use
enable processes cumulatively to use more memory than physically available.
kswapd daemon is used - it checks for physical memory pages not referenced recently, evicts them from
physical memory and stores in a paging file.

Kernel – Conceptual View

memory manager

- **resolves page faults** – for memory addresses not currently mapped to a physical address
when a process accesses a memory address that is not currently mapped to a physical address, the MMU notifies the Linux kernel of the event, which is known as a page fault.
The memory manager subsystem will then resolve the page fault.
 - If page is currently swapped out to the paging file, it is swapped back in, and
 - If memory manager detects an invalid memory address, it notifies the event to process with a signal.
If process is not handling this signal, it is terminated.
- interfacing with memory manager.

using system calls, such as:

- malloc and free - to allocate and free a region of dynamic memory for a process.
- mmap, munmap, msync, and mremap - to maps files into regions in virtual memory.
- mprotect - to change protection on a region of virtual memory
- mlock, mlockall, munlock, and munlockall - super user routines to prevent memory being swapped
- swapon and swapoff - super user routines to add and remove swap files for a system

using kernel calls, such as:

- kmalloc and kfree - to allocate and free a region of dynamic memory for kernel use.
- get_free_page and free_page - to allocate and free physical pages for kernel use.
- verify_area - verify a region of user memory has proper permissions

memory manager architecture

main components, are

- system calls - provides interfaces to memory manager for user space interaction.
- memory mapped files - provides file mapping into memory, avoiding expensive IO calls.
- core memory manager - provides underlying interfaces for implementing memory allocation and free.
- swapfile component - provides access and interfaces to control paging file access.
- architecture-specific modules –provide handling of architecture/hardware specific operations e.g. MMU

Kernel

Conceptual View – Virtual File System

Virtual File System “VFS” is responsible for handling IO polymorphism

across

- multiple hardware devices
- multiple logical file systems
- multiple executable formats

i

- <https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Virtual File System “VFS”

is responsible for handling IO polymorphism

- common interface across logical file systems and hardware devices.
- supports multiple executable file formats such as a.out and ELF.
 - multiple hardware devices
 - multiple logical file systems
 - multiple executable formats

has to

- ensure **high performance** - provide high-speed access to files and
- ensure **data safety and integrity** - enforce policies to not lose or corrupt data
- ensure **data security** - enforce policies of file access and file size quotas to users .

interfacing with memory manager.

- **using system calls**, such as: **open, close, read, write**
- **using kernel calls**, such as: inode and file operations

Virtual File System "VFS"

So, what is an inode?

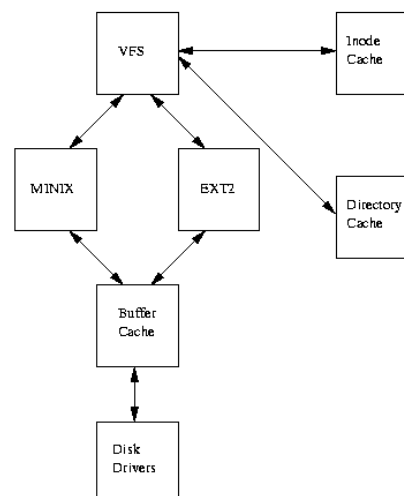
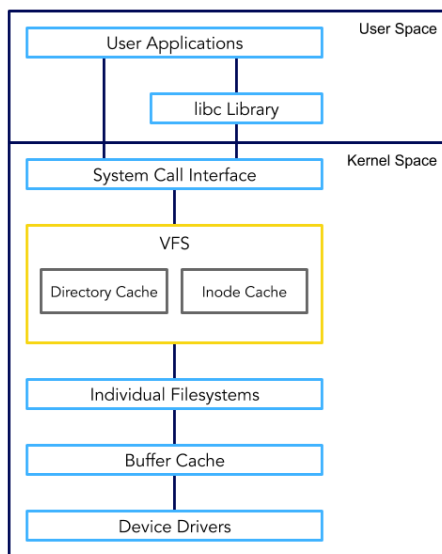
- data structure to stores information about a file - except its name and data.
- contains unique i-node number that is
 - assigned when a file/dir is created.
 - used for accessing the data contents of the file needed.
- used extensively by "ls" command and options.

interfacing with inodes.

- **mknod** create a directory, special file, or regular file
- **create** creates a file in a directory
- **lookup** find a file by name within a directory
- **mkdir, rmdir** create or remove directory
- **readpage, writepage** reads or writes a page of physical memory
- **truncate** set file length to zero
- **permission** check if a user process has permission to execute an operation
- **smap** map a logical file block to a physical device sector
- **bmap** map a logical file block to a physical device block
- **rename** rename a file or directory
- **link, symlink, unlink, readlink, followlink** manage file system links

interfacing with files

- **open, release** open or close a file
- **read, write** read or write to a file
- **select** wait until a file is readable or writeable
- **lseek** move to an offset in the file
- **mmap** maps a region of a file into a user process
- **fsync, fasync** synchronize memory buffers with the physical device
- **readdir** reads file entries pointed to by a directory file
- **ioctl** sets file attributes
- **check_media_change** checks if removable media is removed
- **revalidate** verify all cached information is valid



<https://www.starlab.io/blog/introduction-to-the-linux-virtual-filesystem-vfs-part-i-a-high-level-tour>
<https://tldp.org/LDP/tlk/fs/filesystem.html>

Virtual File System “VFS”

Architecture

- **system call interface**, provides
 - virtual file system services to the user space
 - logical structure for the information stored in a storage medium
- **logical file system**
 - several logical file system types, such as ext2 and fat.
 - all files appear same to the user since i-node is used to hide logical file system details.
 - corresponding fs type is stored in the i-node for each file.
 - depending on i-node, operations are carried out for reading or writing to file.
- **device drivers**
 - an important part
 - provides uniform interface to hardware devices.
 - typically two types of devices:
 - char devices is accessed sequentially, such as a mouse, serial port.
 - block device is accessed randomly, and data is read or written in block, for example, a disk drive.
- device drivers use file interface abstraction.
 - each device is accessed as a file
 - using a special file, known as the device file
 - provides implementation of hardware-specific code to customize file interface abstraction.

Kernel

Conceptual View – IPC

Inter Process Communication “IPC” provides

- sharing resources
- data synchronization, and
- data exchange

- <https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Inter Process Communication “IPC” provides

- sharing resources
- data synchronization, and
- data exchange

provided using system call interface.

includes

- Process IPC : signals
- File IPC: pipes,
- Posix / System V IPC: shared memory, semaphores, message queues,
- Network IPC sockets,
- Kernel IPC: wait queues,
- each of above is in a different category in regard to IPC and what it may be used for.

Kernel

Conceptual View – Network

Network Connectivity

- supports networking.
- implements network protocols such as TCP and IP through hardware-independent code.
- implements network card drivers; through hardware-specific code.

- <https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Network Connectivity

- supports networking.
- implements network protocols such as TCP and IP through hardware-independent code.
- implements network card drivers; through hardware-specific code.

Kernel

Conceptual View – Device Tree

- for a kernel to manage hardware resources, it needs to know which resources are available to the embedded system, looking back to the hardware description and things such as I/O devices and memory.
- two ways for kernel to retrieve this information.
 - hardcoded into the kernel binary code.
 - provide information to kernel via bootloader that provides device tree blob - DTB.

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Device Tree

- for a kernel to manage hardware resources, it needs to know which resources are available to the embedded system, looking back to the hardware description and things such as I/O devices and memory.
- two ways for kernel to retrieve this information.
 - hardcoded into the kernel binary code.
each modification to the hardware definition requires recompiling the source code.
 - provide information to kernel via bootloader that provides device tree blob - DTB.
DTB file is produced from a device tree source DTS.
definition of the hardware can be changed much more easily, as only the DTS requires recompilation.
saves a large amount of time.
- In Arm architecture, the DTS files are typically located in arch/arm/boot/dts or arch/arm64/boot/dts.
 - .dts files are for board-level definitions and the .dtsi files are for included files.
 - device tree compiler is a tool that compiles the source into a binary form: the DTB.
DTB is then loaded by the bootloader and parsed by the kernel at boot time.
- device tree files are not monolithic,
so they can be split into several files and “included” into a single large source tree file.

Kernel

Conceptual View – Linux Kernel Modules

kernel module

- loaded at runtime,
- avoids kernel recompile and reboot. saves time.
- dynamically extends kernel functionality.

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Kernel Module,

- is loaded at runtime,
- avoids kernel recompile and reboot. saves time.
- dynamically extends kernel functionality.

A device driver

- provides an I/O-device specific implementation of the VFS abstraction.
- can be linked with the Linux kernel and executed at system bootstrap, or
- as a kernel module which is loaded at runtime

Linux kernel module utilities include:

- insmod inserts a module into the kernel
- rmmod removes a module from the kernel
- lsmod lists modules loaded in the kernel

functions in kernel module include

- initialization function called by the execution of insmod
- clean-up function called by the execution of rmmod

Kernel Module,

- initialization function
startup
- cleanup function
teardown

Kernel

Conceptual View – Interfacing with Drivers

Device Drivers,

- provide mechanism for interacting with underlying hardware
- need to know how device drivers fit into overall architecture and how to access them from user space; since a system could have some new pieces of hardware.

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Device Drivers,

- provide mechanism for interacting with underlying hardware
- need to know how device drivers fit into overall architecture and how to access them from user space; since a system could have some new pieces of hardware.
- In many cases, device drivers are already provided and so, there is no reason for any new kernel code.
- There is plenty of documentation on how to write device driver, but few indicate why and choices available.
- this is not dedicated to writing kernel device drivers, and that the information given here is to help navigate the territory.
- There are many good books and articles that will help you to write device drivers

Role of Device Drivers.

- the kernel has
 - encapsulates hardware interfaces of a system and present them in a consistent manner to user space.
 - frameworks designed to make it easy to write a device driver, which is the piece of code that mediates between the kernel above and the hardware below.
- A device driver could control physical devices such as a UART or an MMC controller, or represent a virtual device such as the null device (/dev/null) or a ramdisk.
- One driver may control multiple devices of the same kind.
- Kernel device driver code runs at a high privilege level, as does the rest of the kernel.
- It has full access to the processor address space and hardware registers.
- It can handle interrupts and DMA transfers.
- It can also make use of sophisticated kernel infrastructure for synchronization and memory management.
- However, you should be aware that there is a downside to this; if something goes wrong in a buggy driver, it can go really wrong and bring the system down.
- Consequently, there is a principle that device drivers should be as simple as possible by just providing information to applications where the real decisions are made.
- There is no policy in the kernel.
It is the responsibility of user space to set the policy that governs the overall behavior of the system.
- In Linux, there are three main types of device driver:
 - **Character**: This is for an unbuffered I/O with a rich range of functions and a thin layer between the application code and the driver. It is the first choice when implementing custom device drivers.
 - **Block**: This has an interface tailored for block I/O to and from mass storage devices. There is a thick layer of buffering designed to make disk reads and writes as fast as possible, which makes it unsuitable for anything else.
 - **Network**: This is similar to a block device but is used for transmitting and receiving network packets rather than disk blocks.
- There is also a fourth type organized by interface: SCSI, USB, PCI, GPIO

Kernel

Conceptual View – Interfacing with Drivers 2

Char Drivers,

- provide mechanism for interacting with underlying hardware
- need to know how device drivers fit into overall architecture and how to access them from user space; since a system could have some new pieces of hardware.

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Char Drivers,

- char devices are identified in user space by a special file called a device node - mapped to a device driver using the major and minor numbers associated with it.
- Broadly speaking, the major number maps the device node to a particular device driver, while the minor number tells the driver which interface is being accessed.
- list of standard major, minor numbers is found in the kernel documentation in Documentation/devices.txt.
- device nodes can be created in several ways:
 - **devtmpfs**: The device node is created when the device driver registers a new device interface using a base name supplied by the driver and an instance number.
 - **udev** or **mdev** (without devtmpfs): Essentially similar to devtmpfs, except that a user space daemon program extracts the device name from sysfs and creates the node.
 - **mknod**: If using static device nodes, they are created manually using mknod.
- both major and minor numbers are 8-bit numbers in the range 0 to 255.
- from Linux 2.6 onward, the major number is 12 bits long, from 1 to 4k-1, and the minor number is 20 bits, from 0 to 1M-1.
- When a char device node is opened, the kernel checks whether the major and minor numbers fall into a range registered by a character device driver. If so, the call is passed to driver; otherwise, the open call fails.
- The device driver also extract the minor number to find out which hardware interface to use.
- A device driver is not the same as a file: the APIs change the state of the device.

Kernel

Conceptual View – Interfacing with Drivers 3

Block Device,

- is accessed randomly, and
- data is read or written in block

e.g. a disk drive.

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Block Device,

- is accessed randomly, and data is read or written in block e.g. a disk drive.
- also have major and minor numbers.
Though char and block devices are both identified using major and minor numbers, they are in different namespaces. A char device with a major# 100 is unrelated to a similar block device with major# 100.
- With block devices, major# identifies the device driver and the minor# identifies the partition.
- mmcblk0 is the microSD card slot, it has a card with two partitions, and mmcblk1 is the eMMC chip, which also has two partitions.
- major number for the MMC block driver is 179.
- The minor numbers are used in ranges to identify different physical MMC devices, and the partitions of the storage medium that are on that device.
- For a MMC driver, the ranges are eight minor numbers per device: the minor numbers from 0 to 7 are for the first device, the numbers from 8 to 15 are for the second, and so on.
- Within each range, the first minor number represents the entire device as raw sectors, and the others represent up to seven partitions.
- On eMMC chips, there are two 128 KiB areas of memory reserved for use by a bootloader.
- These are represented as two devices known as mmcblk1boot0 and mmcblk1boot1, and they have minor numbers of 16 and 24, respectively.
- Also, for a SCSI disk driver, known as sd, which is used to control a range of disks that use SCSI command set, which includes SCSI, SATA, USB mass storage, and Universal Flash Storage (UFS).
- It has the major number 8 and ranges of 16 minor numbers per interface (or disk).

Block Device,

- The minor numbers from 0 to 15 are for the first interface with device nodes named sda up to sda15, the numbers from 16 to 31 are for the second disk with device nodes sdb up to sdb15, and so on.
- This continues up to the 16th disk from 240 to 255 with the node name sdp.
- There are other major numbers reserved for them because SCSI disks are so popular.
- Both the MMC and SCSI block drivers expect to find a partition table at the start of the disk.
- The partition table is created using utilities such as fdisk, sfdisk, and parted.
- A user space program can open and interact with a block device directly via the device node.
- This is not a common thing to do, though, and is usually only done to perform administrative operations such as creating partitions, formatting a partition with a filesystem, and mounting.
- Once the filesystem has been mounted, interact with the block device indirectly through the files in that filesystem.
- Most block devices will have a kernel driver that works, so rarely need to write our own.
- The same goes for network devices.

Just like a filesystem abstracts the details of a block device, the network stack eliminates the need to interact directly with a network device.

Kernel

Conceptual View – Interfacing with Drivers 3

Network Device,

- Network devices are not accessed through device nodes, they do not have major and minor numbers.
- Instead, a network device is allocated a name by the kernel, based on a string and an instance number.

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Network Device,

- Network devices are not accessed through device nodes, and they do not have major and minor numbers. Instead, a network device is allocated a name by the kernel, based on a string and an instance number.
- a network device could be named net0 the first time it is called, net1 the second time, and so on. More commonly, names include lo, eth0, and wlan0.
- However, also, this is the name it starts off with; since the device managers, such as udev, may change it to something different later on.
- Usually, the network interface name is only used when configuring the network using utilities, such as ip and ifconfig, to establish a network address and route.
- Thereafter, network driver interaction is done indirectly by opening sockets and letting the network layer decide how to route them to the right interface.
- it is possible to access network devices directly from the user space by creating a socket and using the ioctl commands listed in include/linux/sockios.h.

Kernel

Finding out about drivers

At run time,

- to know which device drivers
- have been loaded and what state they are in

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

At run-time,

- know which device drivers have been loaded and what state they are in.
- list the character and block device drivers that are currently loaded and active by reading `/proc/devices`:
`cat /proc/devices`
- For each driver, you can see the major number and the base name. However, this does not tell you how many devices each driver is attached to. It only shows a device, but it gives no clue as to whether it is attached to four real serial ports.
- network devices do not appear in the list, because they do not have device nodes. Instead, use tools such as `ifconfig` or `ip` to get a list of network devices:
`# ip link show`
- to find out about devices attached to USB or PCI buses use the `lsusb` and `lspci` commands.

Kernel sysfs

At run time,

- to know which device drivers
- have been loaded and what state they are in

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

sysfs,

- is a representation of kernel objects, attributes, and relationships.
A kernel object is a directory, an attribute is a file, and a relationship is a symbolic link from one object to another.
- # ls /sys
- block class devices fs module
- bus dev firmware kernel power

The devices – /sys/devices

- provides kernel' view of devices discovered since boot and how they are connected to each other.
It is organized at the top level by the system bus, so it varies from one system to another.
- # ls /sys/devices
- platform software system tracepoint virtual
- three directories are present on all systems:
 - **system/**: This contains devices at the heart of the system, including CPUs and clocks.
 - **virtual/**: This contains devices that are memory-based. You will find the memory devices that appear as /dev/null, /dev/random, and /dev/zero in virtual/mem. You will find the loopback device, lo, in virtual/net.
 - **platform/**: This is a catch-all for devices that are not connected via a conventional hardware bus. This may be almost everything on an embedded device.

/sys/class and /sys/block offer two different views of the devices.

The drivers – /sys/class

- view of the device drivers presented by type. it is a software view rather than a hardware view. Each subdirectory represents a class of driver and is implemented by a component of driver framework. e.g. UART devices are managed by the tty layer, under /sys/class/tty. Likewise, network devices are in /sys/class/net, input devices such as the keyboard, the touchscreen, and mouse in /sys/class/input, and so on.
- a symbolic link in each subdirectory under a type of device pointing to representation in /sys/device.
- each directory is a representation of the kernel object associated with an instance of a device interface. within each of these directories, are the attributes of the object, represented as files, and the relationships with other objects, represented by links:
ls /sys/class/tty/ttyAMA0
close_delay flags line uartclk
closing_wait io_type port uevent
custom_divisor iomem_base power xmit_fifo_size
dev iomem_reg_shift subsystem
device irq type
- The link called device points to the hardware object for the device while the link called subsystem points back to the parent subsystem, /sys/class/tty. The remaining directory entries are attributes. Some are specific to a serial port, such as xmit_fifo_size, while others apply to many types of device, such as the interrupt number, irq, and the device number, dev. Some attribute files are writable and allow tuning of parameters in the driver at runtime.
- The dev attribute is particularly interesting. It shows
cat /sys/class/tty/ttyAMA0/dev
204:64
- These are major# and minor# for this device and, is created when the driver registers this interface. It is from this file that udev and mdev find the major and minor numbers of the device driver.

The block drivers – /sys/block

- There is one more view of the device model that is important: the block driver view in /sys/block. There is a subdirectory for each block device. e.g.
ls /sys/block
loop0 loop4 mmcblk0 ram0 ram12 ram2 ram6
loop1 loop5 mmcblk1 ram1 ram13 ram3 ram7
loop2 loop6 mmcblk1boot0 ram10 ram14 ram4 ram8
loop3 loop7 mmcblk1boot1 ram11 ram15 ram5 ram9
- In mmcblk1, which is the eMMC chip on this board, see the attributes of the interface and the partitions within it:
ls /sys/block/mmcblk1
alignment_offset ext_range mmcblk1p1 ro
bdi force_ro mmcblk1p2 size
capability holders power slaves
dev inflight queue stat
device mmcblk1boot0 range subsystem
discard_alignment mmcblk1boot1 removable uevent
- The conclusion, then, is that you can learn a lot about the devices (the hardware) and the drivers (the software) that are present on a system by reading sysfs.

Kernel

Device Drivers in User Space

Is a kernel-level device driver really necessary?

No need to write a line of kernel code.

- Many generic device drivers are available for many common types of devices - that interact with hardware directly from user space,
- User space code is certainly easier to write and debug.
- It is also not covered by the GPL.

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Is a kernel-level device driver really necessary? – No need to write a line of kernel code

- Many generic device drivers are available - that interact with hardware directly from user space.
- User space code is certainly easier to write and debug.
- It is also not covered by the GPL, but that is a good reason to do it this way.
- These drivers fall into two broad categories:
 - control through files in sysfs, including GPIO and LEDs, and
 - serial buses that expose a generic interface through a device node, such as I2C.

GPIO

General-Purpose Input/Output (GPIO) is the simplest form of digital interface

- It provides direct access to individual hardware pins, each of which can be in one of two states: high or low.
- In most cases, a GPIO pin can be configured to be either an input or an output.
- A group of GPIO pins can be used for high-level interfaces .. I2C, SPI by software emulation, a technique that is called bit banging. main limitation is speed, accuracy of loops and #CPU cycles to dedicate to them. Generally it is hard to achieve timer accuracy better than a msec unless a real-time kernel is configured.
- Common GPIO use cases are reading push buttons, digital sensors and controlling LEDs, motors, relays.
- Most SoCs have a lot of GPIO bits, grouped together in GPIO registers, usually 32 bits per register.
- On-chip GPIO bits are routed to GPIO pins on the chip package via a multiplexer, known as a pin mux.
- Also GPIO pins can be off-chip in power management chip, or in dedicated GPIO extenders, via I2C or SPI.
- gpiolib - a kernel subsystem, not actually a library but for infrastructure GPIO drivers to provide IO entry-pt.
- Details of gpiolib are in kernel source in Documentation/gpio, and driver code is in drivers/gpio.
- Applications interact with gpiolib through files in the /sys/class/gpio directory.

- As an example of what is seen on a typical embedded board a BeagleBone Black):

```
# ls /sys/class/gpio
export gpiochip0 gpiochip32 gpiochip64 gpiochip96 unexport
```
- directories are named gpiochip0 .. gpiochip96 represent four GPIO registers, each with 32 GPIO bits.
- In each one of the gpiochip directories, it will show the following:

```
# ls /sys/class/gpio/gpiochip96
base label ngpio power subsystem uevent
```
- file named base contains the first GPIO pin in register, while ngpio contains number of bits in the register.
- So, if gpiochip96/base is 96 and gpiochip96/ngpio is 32, then it shows that it contains GPIO bits 96 to 127.
- Also, there could be a gap between the last GPIO in one register and the first GPIO in the next.
- To control a GPIO bit from user space, export from kernel space, by writing GPIO# to /sys/class/gpio/export.
e.g. the process for GPIO 53, which is wired to user LED 0 on the BeagleBone Black:

```
# echo 53 > /sys/class/gpio/export
# ls /sys/class/gpio
export gpio53 gpiochip0 gpiochip32 gpiochip64 gpiochip96 unexport
```
- a new directory, gpio53, is created, which contains these files needed to control the pin:
Note: If GPIO bit is already claimed by kernel, it cannot be exported in this way.

```
# ls /sys/class/gpio/gpio53
active_low direction power uevent device edge subsystem value
```
- The pin begins as an input, and to change it to an output, write out to the **direction** file.
The file **value** contains the current state of the pin .. 0 for low and 1 for high. change state by writing 0 or 1.
- Sometimes, the meaning of low and high is reversed in hardware, so writing 1 to active_low inverts the meaning of value so that a low voltage is reported as 1 and a high voltage is reported as 0.
- Remove a GPIO from user space control by writing the GPIO number to /sys/class/gpio/unexport.

Handling interrupts from GPIO

- a GPIO input can be configured to generate interrupt upon state change .. allows waiting for interrupt rather than polling in an inefficient software loop.
- If GPIO bit can generate interrupts, a file called edge exists. Initially, it has value none, meaning that it does not generate interrupts. To enable interrupts, set it to one of these values:
rising: Interrupt on rising edge
falling: Interrupt on falling edge
both: Interrupt on both rising and falling edges
none: No interrupts (default)
- To wait for a falling edge on GPIO 48, first enable the interrupts:

```
# echo 48 > /sys/class/gpio/export
# echo falling > /sys/class/gpio/gpio48/edge
```
- To wait for an interrupt from the GPIO, follow these steps:

```
// 1. First, call epoll_create to create the epoll notification facility:
int ep;
ep = epoll_create(1);
// 2. Next, open the GPIO and read out its initial value:
int f;
int n;
char value[4];
f = open("/sys/class/gpio/gpio48/value", O_RDONLY | O_NONBLOCK);
[...]
n = read(f, &value, sizeof(value));
if (n > 0) {
    printf("Initial value value=%c\n",
        value[0]);
    lseek(f, 0, SEEK_SET);
```

```

}
// 3. Call epoll_ctl to register the GPIO's file descriptor with POLLPRI as the event:
struct epoll_event ev, events;
ev.events = EPOLLPRI;
ev.data.fd = f;
int ret;
ret = epoll_ctl(ep, EPOLL_CTL_ADD, f, &ev);
4. Lastly, wait for an interrupt using the epoll_wait function:
while (1) {
    printf("Waiting\n");
    ret = epoll_wait(ep, &events, 1, -1);
    if (ret > 0) {
        n = read(f, &value, sizeof(value));
        printf("Button pressed: value=%c\n", value[0]);
        lseek(f, 0, SEEK_SET);
    }
}

```

- While select and poll could be used to handle interrupts, unlike those other two system calls, the performance of epoll does not degrade rapidly as the number of file descriptors being monitored increases.

LEDs

- Like GPIOs, LEDs are accessible from sysfs.
- The interface, however, is noticeably different.
- LEDs are often controlled through a GPIO pin, but another kernel subsystem offers more specialized control, such as ability to set brightness, or to handle LEDs connected in other ways than a simple GPIO pin, or to trigger the LED on an event, such as block device access or a heartbeat, to show that device is working. Configure kernel with CONFIG_LEDS_CLASS option and with the LED trigger actions that are appropriate.
- More information is in Documentation/leds/, and the drivers are in drivers/leds/.
- As with GPIOs, LEDs are controlled through an interface in sysfs in the /sys/class/leds directory.
- In the case of the BeagleBone Black, the names of the LEDs are encoded in the device tree in the form of devicename:colour:function, as shown:

```

# ls /sys/class/leds
beaglebone:green:heartbeat beaglebone:green:usr2
beaglebone:green:mmc0 beaglebone:green:usr3

```

looking at the attributes of one of the LEDs:

```

# cd /sys/class/leds/beaglebone:green\:usr2
# ls
brightness max_brightness subsystem uevent device power trigger

```

- The brightness file controls the LED as a number between 0 (off) and max_brightness (fully on).
- If the LED doesn't support intermediate brightness, any non-zero value turns it on.
- The file called trigger lists the events to trigger the LED to turn on, and is implementation-dependent.

Here is an example:

```

# cat trigger
none mmc0 mmc1 timer oneshot heartbeat backlight gpio
[cpu0]
default-on

```

- trigger currently selected is shown in square brackets, and change it by writing another trigger to the file. To control the LED entirely through brightness, select none. To set trigger to timer, two extra files will appear that allow setting the on and off times in milliseconds:


```

# echo timer > trigger
# ls
brightness delay_on max_brightness subsystem uevent delay_off device power trigger

```

```
# cat delay_on
500
# cat
/sys/class/leds/beaglebone:green:heartbeat/delay_off
500
```

If the LED has on-chip timer hardware, the blinking takes place without interrupting the CPU.

I2C

I2C is a simple low speed 2-wire bus that is common on embedded boards, typically used to access peripherals that are not on the SoC, such as display controllers, camera sensors, GPIO extenders, and so on.

There is a related standard known as system management bus (SMBus) that is found on PCs, which is used to access temperature and voltage sensors.

SMBus is a subset of I2C.

I2C is a master-slave protocol, with the master being one or more hostcontrollers on the SoC.

Slaves have a 7-bit address assigned by the manufacturer (read the data sheet), allowing up to 128 nodes per bus, but 16 are reserved, so

only 112 nodes are allowed in practice.

The master may initiate read or write transactions with one of the slaves.

Frequently, the first byte is used to specify a register on the slave, while the remaining bytes are the data that's read from or written to that register.

There is one device node for each host controller; for example, this SoC has four:

```
# ls -l /dev/i2c*
crw-rw---- 1 root i2c 89, 0 Jan 1 00:18 /dev/i2c-0
crw-rw---- 1 root i2c 89, 1 Jan 1 00:18 /dev/i2c-1
crw-rw---- 1 root i2c 89, 2 Jan 1 00:18 /dev/i2c-2
crw-rw---- 1 root i2c 89, 3 Jan 1 00:18 /dev/i2c-3
```

The device interface provides a series of ioctl commands that query the host controller and send the read and write commands to I2C slaves.

There is a package named i2c-tools, which uses this interface to provide basic command-line tools to interact with I2C devices.

The tools are as follows:

i2cdetect: This lists the I2C adapters and probes the bus.

i2cdump: This dumps data from all the registers of an I2C peripheral.

i2cget: This reads data from an I2C slave.

i2cset: This writes data to an I2C slave.

The i2c-tools package is available in Buildroot and the Yocto Project, as well as most mainstream distributions.

So, as long the address and protocol of the slave is known, writing a user space program to talk to the device is straightforward.

The example that follows shows how to read the first four bytes from the AT24C512B EEPROM, which is mounted on the BeagleBone Black on

I2C bus 0.

It has a slave address of 0x50:

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/ioctl.h>
```

```

#include <linux/i2c-dev.h>
#define I2C_ADDRESS 0x50
int main(void)
{
    int f;
    int n;
    char buf[10];
    f = open("/dev/i2c-0", O_RDWR);
    /* Set the address of the i2c slave device */
    ioctl(f, I2C_SLAVE, I2C_ADDRESS);
    /* Set the 16-bit address to read from to 0 */
    buf[0] = 0; /* address byte 1 */
    buf[1] = 0; /* address byte 2 */
    n = write(f, buf, 2);
    /* Now read 4 bytes from that address */
    n = read(f, buf, 4);
    printf("0x%x 0x%x0 0x%x 0x%x\n",
        buf[0], buf[1], buf[2], buf[3]);
    close(f);
    return 0;
}

```

This program is like `i2cget` except that the address and register bytes being read from are both hardcoded rather than passed in as arguments.

Use `i2cdetect` to discover the addresses of any peripherals on an I2C bus.

`i2cdetect` can leave I2C peripherals in a bad state or lock up the bus, so it's good practice to reboot after using it.

A peripheral's data sheet shows what the registers map to.

With that information, use `i2cset` to write to its registers over I2C.

These I2C commands can easily be converted into a library of C functions for interfacing with the peripheral.

There is more information about the Linux implementation of I2C in `Documentation/i2c/dev-interface`.

The host controller drivers are in `drivers/i2c/busses`.

Another popular communication protocol is the Serial Peripheral Interface (SPI), which utilizes a 4-wire bus.

SPI

The SPI bus is similar to I2C but is a lot faster by up to tens of MHz.

The interface uses four wires with separate send and receive lines, which allow it to operate in full duplex.

Each chip on the bus is selected with a dedicated chip select line.

It is commonly used to connect to touchscreen sensors, display controllers, and serial NOR flash devices.

As with I2C, it is a master-slave protocol, with most SoCs implementing one or more master host controllers.

There is a generic SPI device driver, which is enabled through the `CONFIG_SPI_SPIDEV` kernel configuration.

It creates a device node for each SPI controller, to allow access to SPI chips from the user space. The device nodes are named `spidev[bus].[chip select]`:


```
# ls -l /dev/spi*
```

```
crw-rw---- 1 root root 153, 0 Jan 1 00:29
```

```
/dev/spidev1.0
```

For examples of using the spidev interface, please refer to the example code in Documentation/spi. So far, all the device drivers have longstanding upstream support within the Linux kernel.

Because these device drivers are all generic (GPIO, LEDs, I2C, and SPI), accessing them from the user space is straightforward.

At some point, there will be a piece of hardware that lacks a compatible kernel device driver.

That hardware may be the centerpiece of a product (for example, Lidar, SDR, and so on).

There may also be an FPGA in-between the SoC and this hardware.

Under these circumstances, there is no other recourse than to write own kernel module(s). There are generic device drivers for many common types of devices that interact with hardware directly from user space, without having to write a line of kernel code.

User space code is certainly easier to write and debug.

It is also not covered by the GPL, but that is a good reason to do it this way.

These drivers fall into two broad categories:

control through files in sysfs, including GPIO and LEDs, and

serial buses that expose a generic interface through a device node, such as I2C.

GPIO

General-Purpose Input/Output (GPIO) is the simplest form of digital interface since it gives you direct access to individual hardware pins, each of which can be in one of two states: either high or low.

In most cases, a GPIO pin can be configured to be either an input or an output.

a group of GPIO pins can even be used to create higher level interfaces such as I2C or SPI by manipulating each bit in software, a technique that is called bit banging.

The main limitation is the speed and accuracy of the software loops and the number of CPU cycles to dedicate to them.

Generally speaking, it is hard to achieve timer accuracy better than a millisecond unless a real-time kernel is configured.

More common use cases for GPIO are for reading push buttons and digital sensors and controlling LEDs, motors, and relays.

Most SoCs have a lot of GPIO bits, grouped together in GPIO registers, usually 32 bits per register. On-chip GPIO bits are routed through to GPIO pins on the chip package via a multiplexer, known as a pin mux.

There may be additional GPIO pins available off-chip in the power management chip, and in dedicated GPIO extenders, connected through I2C or SPI buses.

All this diversity is handled by a kernel subsystem known as gpiolib, which is not actually a library but the infrastructure GPIO drivers used to expose I/O in a consistent way.

Details about the implementation of gpiolib are in the kernel source in Documentation/gpio, and the code for the drivers themselves is in drivers/gpio.

Applications can interact with gpiolib through files in the /sys/class/gpio directory.

As an example of what is seen on a typical embedded board a BeagleBone Black):

```
# ls /sys/class/gpio
```

```
export gpiochip0 gpiochip32 gpiochip64 gpiochip96 unexport
```

The directories named gpiochip0 through to gpiochip96 represent four GPIO registers, each with 32 GPIO bits.

In each one of the gpiochip directories, it will show the following:

```
# ls /sys/class/gpio/gpiochip96
```

base label ngpio power subsystem uevent

The file named base contains the number of the first GPIO pin in the register, while ngpio contains the number of bits in the register.

In this case, gpiochip96/base is 96 and gpiochip96/ngpio is 32, which tells you that it contains GPIO bits 96 to 127.

It is possible for there to be a gap between the last GPIO in one register and the first GPIO in the next.

To control a GPIO bit from user space, you first have to export it from kernel space, which you can do by writing the GPIO number to

/sys/class/gpio/export.

This example shows the process for GPIO 53, which is wired to user LED 0 on the BeagleBone Black:

```
# echo 53 > /sys/class/gpio/export
```

```
# ls /sys/class/gpio
```

```
export gpio53 gpiochip0 gpiochip32 gpiochip64
```

```
gpiochip96 unexport
```

Now, there is a new directory, gpio53, which contains the files needed to control the pin.

If the GPIO bit is already claimed by the kernel, it cannot be exported in this way.

The gpio53 directory contains these files:

```
# ls /sys/class/gpio/gpio53
```

```
active_low direction power uevent device edge subsystem value
```

The pin begins as an input.

To change it to an output, write out to the direction file.

The file value contains the current state of the pin, which is 0 for low and 1 for high.

If it is an output, change the state by writing 0 or 1 to value.

Sometimes, the meaning of low and high is reversed in hardware, so writing 1 to active_low inverts the meaning of value so that a low voltage is reported as 1 and a high voltage is reported as 0.

Remove a GPIO from user space control by writing the GPIO number to /sys/class/gpio/unexport.

Handling interrupts from GPIO

In many cases, a GPIO input can be configured to generate an interrupt when it changes state, which allows waiting for the interrupt rather than polling in an inefficient software loop.

If the GPIO bit can generate interrupts, a file called edge exists.

Initially, it has the value called none, meaning that it does not generate interrupts.

To enable interrupts, set it to one of these values:

rising: Interrupt on rising edge

falling: Interrupt on falling edge

both: Interrupt on both rising and falling edges

none: No interrupts (default)

To wait for a falling edge on GPIO 48, first enable the interrupts:

```
# echo 48 > /sys/class/gpio/export
```

```
# echo falling > /sys/class/gpio/gpio48/edge
```

To wait for an interrupt from the GPIO, follow these steps:

1. First, call epoll_create to create the epoll notification facility:

```
int ep;
```

```
ep = epoll_create(1);
```

2. Next, open the GPIO and read out its initial value:

```
int f;
```

```
int n;
```

```

char value[4];
f = open("/sys/class/gpio/gpio48/value", O_RDONLY | O_NONBLOCK);
[...]
n = read(f, &value, sizeof(value));
if (n > 0) {
    printf("Initial value value=%c\n",
        value[0]);
    lseek(f, 0, SEEK_SET);
}
3. Call epoll_ctl to register the GPIO's file descriptor with POLL_PRI as
the event:
struct epoll_event ev, events;
ev.events = EPOLL_PRI;
ev.data.fd = f;
int ret;
ret = epoll_ctl(ep, EPOLL_CTL_ADD, f, &ev);
4. Lastly, wait for an interrupt using the epoll_wait function:
while (1) {
    printf("Waiting\n");
    ret = epoll_wait(ep, &events, 1, -1);
    if (ret > 0) {
        n = read(f, &value, sizeof(value));
        printf("Button pressed: value=%c\n",
            value[0]);
        lseek(f, 0, SEEK_SET);
    }
}

```

While select and poll could be used to handle interrupts, unlike those other two system calls, the performance of epoll does not degrade rapidly as the number of file descriptors being monitored increases.

Like GPIOs, LEDs are accessible from sysfs.

The interface, however, is noticeably different.

LEDs

LEDs are often controlled through a GPIO pin, but there is another kernel subsystem that offers more specialized control that's specific for this purpose.

LEDs kernel subsystem adds the ability to set brightness, should the LED have that ability, and it can handle LEDs connected in other ways than a simple GPIO pin.

It can be configured to trigger the LED on an event, such as block device access or just a heartbeat, to show that the device is working. Configure the kernel with the CONFIG_LEDS_CLASS option and with the LED trigger actions that are appropriate.

There is more information on Documentation/leds/, and the drivers are in drivers/leds/.

As with GPIOs, LEDs are controlled through an interface in sysfs in the /sys/class/leds directory.

In the case of the BeagleBone Black, the names of the LEDs are encoded in the device tree in the form of

devicename:colour:function, as shown here:

```
# ls /sys/class/leds
```

```
beaglebone:green:heartbeat beaglebone:green:usr2
```

```
beaglebone:green:mmc0 beaglebone:green:usr3
```

looking at the attributes of one of the LEDs:

```
# cd /sys/class/leds/beaglebone\:green\:usr2
```

```
# ls
```

```
brightness max_brightness subsystem uevent device power trigger
```

Note that leading backslashes are required by the shell to escape the colons in the path.

The brightness file controls the brightness of the LED and can be a number between 0 (off) and max_brightness (fully on).

If the LED doesn't support intermediate brightness, any non-zero value turns it on.

The file called trigger lists the events that trigger the LED to turn on.

The list of triggers is implementation-dependent.

Here is an example:

```
# cat trigger
```

```
none mmc0 mmc1 timer oneshot heartbeat backlight gpio
```

```
[cpu0]
```

```
default-on
```

The trigger currently selected is shown in square brackets.

Change it by writing one of the other triggers to the file.

To control the LED entirely through brightness, select none.

To set trigger to timer, two extra files will appear that allow setting the on and off times in milliseconds:

```
# echo timer > trigger
```

```
# ls
```

```
brightness delay_on max_brightness subsystem uevent
```

```
delay_off device power trigger
```

```
# cat delay_on
```

```
500
```

```
# cat
```

```
/sys/class/leds/beaglebone:green:heartbeat/delay_off
```

```
500
```

If the LED has on-chip timer hardware, the blinking takes place without interrupting the CPU.

I2C

I2C is a simple low speed 2-wire bus that is common on embedded boards, typically used to access peripherals that are not on the SoC, such as display controllers, camera sensors, GPIO extenders, and so on.

There is a related standard known as system management bus (SMBus) that is found on PCs, which is used to access temperature and voltage sensors.

SMBus is a subset of I2C.

I2C is a master-slave protocol, with the master being one or more host controllers on the SoC.

Slaves have a 7-bit address assigned by the manufacturer (read the data sheet), allowing up to 128 nodes per bus, but 16 are reserved, so only 112 nodes are allowed in practice.

The master may initiate read or write transactions with one of the slaves.

Frequently, the first byte is used to specify a register on the slave, while the remaining bytes are the data that's read from or

written to that register.

There is one device node for each host controller; for example, this SoC has four:

```
# ls -l /dev/i2c*
```

```
crw-rw---- 1 root i2c 89, 0 Jan 1 00:18 /dev/i2c-0
```

```
crw-rw---- 1 root i2c 89, 1 Jan 1 00:18 /dev/i2c-1
crw-rw---- 1 root i2c 89, 2 Jan 1 00:18 /dev/i2c-2
crw-rw---- 1 root i2c 89, 3 Jan 1 00:18 /dev/i2c-3
```

The device interface provides a series of ioctl commands that query the host controller and send the read and write commands to I2C slaves.

There is a package named i2c-tools, which uses this interface to provide basic command-line tools to interact with I2C devices.

The tools are as follows:

i2cdetect: This lists the I2C adapters and probes the bus.

i2cdump: This dumps data from all the registers of an I2C peripheral.

i2cget: This reads data from an I2C slave.

i2cset: This writes data to an I2C slave.

The i2c-tools package is available in Buildroot and the Yocto Project, as well as most mainstream distributions.

So, as long the address and protocol of the slave is known, writing a user space program to talk to the device is straightforward.

The example that follows shows how to read the first four bytes from the AT24C512B EEPROM, which is mounted on the BeagleBone Black on I2C bus 0.

It has a slave address of 0x50:

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <linux/i2c-dev.h>
#define I2C_ADDRESS 0x50
int main(void)
{
    int f;
    int n;
    char buf[10];
    f = open("/dev/i2c-0", O_RDWR);
    /* Set the address of the i2c slave device */
    ioctl(f, I2C_SLAVE, I2C_ADDRESS);
    /* Set the 16-bit address to read from to 0 */
    buf[0] = 0; /* address byte 1 */
    buf[1] = 0; /* address byte 2 */
    n = write(f, buf, 2);
    /* Now read 4 bytes from that address */
    n = read(f, buf, 4);
    printf("0x%x 0x%x0 0x%x 0x%x\n",
        buf[0], buf[1], buf[2], buf[3]);
    close(f);
    return 0;
}
```

This program is like i2cget except that the address and register bytes being read from are both hardcoded rather than passed in as arguments.

Use i2cdetect to discover the addresses of any peripherals on an I2C bus.

i2cdetect can leave I2C peripherals in a bad state or lock up the bus, so it's good practice to reboot after using it.

A peripheral's data sheet shows what the registers map to.

With that information, use i2cset to write to its registers over I2C.

These I2C commands can easily be converted into a library of C functions for interfacing with the peripheral.

There is more information about the Linux implementation of I2C in Documentation/i2c/dev-interface.

The host controller drivers are in drivers/i2c/busses.

Another popular communication protocol is the Serial Peripheral Interface (SPI), which utilizes a 4-wire bus.

SPI

The SPI bus is similar to I2C but is a lot faster by up to tens of MHz.

The interface uses four wires with separate send and receive lines, which allow it to operate in full duplex.

Each chip on the bus is selected with a dedicated chip select line.

It is commonly used to connect to touchscreen sensors, display controllers, and serial NOR flash devices.

As with I2C, it is a master-slave protocol, with most SoCs implementing one or more master host controllers.

There is a generic SPI device driver, which is enabled through the CONFIG_SPI_SPIDEV kernel configuration.

It creates a device node for each SPI controller, to allow access to SPI chips from the user space.

The device nodes are named spidev[bus].[chip select]:

```
# ls -l /dev/spi*
```

```
crw-rw---- 1 root root 153, 0 Jan 1 00:29
```

```
/dev/spidev1.0
```

For examples of using the spidev interface, please refer to the example code in Documentation/spi.

So far, all the device drivers have longstanding upstream support within the Linux kernel.

Because these device drivers are all generic (GPIO, LEDs, I2C, and SPI), accessing them from the user space is straightforward.

At some point, there will be a piece of hardware that lacks a compatible kernel device driver.

That hardware may be the centerpiece of a product (for example, Lidar, SDR, and so on).

There may also be an FPGA in-between the SoC and this hardware.

Under these circumstances, there is no other recourse than to write own kernel module(s).

```
# cat delay_on
500
# cat
/sys/class/leds/beaglebone:green:heartbeat/delay_off
500
```

- If the LED has on-chip timer hardware, the blinking takes place without interrupting the CPU.

I2C

- I2C is a simple low speed 2-wire bus that is common on embedded boards, typically used to access peripherals that are not on SoC, such as display controllers, camera sensors, GPIO extenders, and so on.
- A related system management bus (SMBus) is found on PCs, to access temperature and voltage sensors. SMBus is a subset of I2C.
- I2C is a master-slave protocol, with the master being one or more hostcontrollers on the SoC.
- Slaves have a 7-bit address assigned by the manufacturer (read the data sheet), allowing up to 128 nodes per bus, but 16 are reserved, so only 112 nodes are allowed in practice.
- The master may initiate read or write transactions with one of the slaves.
- Frequently, the first byte is used to specify a register on the slave, while the remaining bytes are the data that's read from or written to that register.
- There is one device node for each host controller; for example, this SoC has four:

```
# ls -l /dev/i2c*
crw-rw---- 1 root i2c 89, 0 Jan 1 00:18 /dev/i2c-0
crw-rw---- 1 root i2c 89, 1 Jan 1 00:18 /dev/i2c-1
crw-rw---- 1 root i2c 89, 2 Jan 1 00:18 /dev/i2c-2
crw-rw---- 1 root i2c 89, 3 Jan 1 00:18 /dev/i2c-3
```

- a series of ioctl commands to query host controller and send the read and write commands to I2C slaves.
- Also, a package i2c-tools, uses these interfaces for basic command-line tools to interact with I2C devices. The tools are as follows:

i2cdetect: This lists the I2C adapters and probes the bus.

i2cdump: This dumps data from all the registers of an I2C peripheral.

i2cget: This reads data from an I2C slave.

i2cset: This writes data to an I2C slave.

- i2c-tools package is available in Buildroot and the Yocto Project, as well as most mainstream distributions.
- as long address and protocol of slave is known, a user space program to talk to device is straightforward.
- to read first four bytes from AT24C512B EEPROM - mounted on BeagleBone Black on I2C bus 0 - with a slave address of 0x50:

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <linux/i2c-dev.h>
#define I2C_ADDRESS 0x50
int main(void)
{
    int f;
    int n;
    char buf[10];
    f = open("/dev/i2c-0", O_RDWR);
    /* Set the address of the i2c slave device */
    ioctl(f, I2C_SLAVE, I2C_ADDRESS);
    /* Set the 16-bit address to read from to 0 */
    buf[0] = 0; /* address byte 1 */
```

```

buf[1] = 0; /* address byte 2 */
n = write(f, buf, 2);
/* Now read 4 bytes from that address */
n = read(f, buf, 4);
printf("0x%x 0x%x 0x%x 0x%x\n",
buf[0], buf[1], buf[2], buf[3]);
close(f);
return 0;
}

```

- It is like i2cget except that address and register bytes being read are both hardcoded rather than passed in.
- Use i2cdetect to discover the addresses of any peripherals on an I2C bus. However, i2cdetect can leave I2C peripherals in a bad state or lock up the bus, so it's good practice to reboot after using it.
- Using a data sheet to shows the registers it maps to, use i2cset to write to its registers over I2C.
- These I2C commands can easily be converted into a library of C functions for interfacing with peripherals.
- More information about the Linux implementation of I2C is in Documentation/i2c/dev-interface, while the host controller drivers are in drivers/i2c/busses.

SPI

- Another popular communication protocol is the Serial Peripheral Interface (SPI), which utilizes a 4-wire bus.
- It is similar to I2C but is a lot faster by up to tens of MHz.
- The interface uses four wires with separate send and receive lines, which allow it to operate in full duplex.
- Each chip on the bus is selected with a dedicated chip select line.
- It is commonly used to connect to touchscreen sensors, display controllers, and serial NOR flash devices.
- As in I2C, it is a master-slave protocol, with most SoCs implementing one or more master host controllers.
- There is a generic SPI device driver, enabled through the CONFIG_SPI_SPIDEV kernel configuration, that creates a device node for each SPI controller, to allow access to SPI chips from the user space.

The device nodes are named spidev[bus].[chip select]:

```

# ls -l /dev/spi*
crw-rw---- 1 root root 153, 0 Jan 1 00:29
/dev/spidev1.0

```

- For using the spidev interface, example code is in Documentation/spi.
- all these device drivers (GPIO, LEDs, I2C, SPI), have longstanding upstream support within Linux kernel.
- Because these device drivers are all generic accessing them from user space is straightforward.
- At some point, there will be a piece of hardware that lacks a compatible kernel device driver -which may be the centerpiece of a product (for example, Lidar, SDR, and so on).
- There may also be an FPGA in-between the SoC and this hardware.
- Under these circumstances, there is no other recourse than to write own kernel module(s).

Kernel

Writing a kernel device driver

there could be a need write a device driver to access a device.

- Character drivers are the most flexible and should cover 90% needs;
- block drivers are for mass storage, while
- network drivers apply .. if working with a network interface.

The task of writing a kernel driver is complex.

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Writing a kernel device driver

- there could be a need write a device driver to access a device.
- Character drivers are the most flexible and should cover 90% needs; block drivers are for mass storage, while network drivers apply .. if working with a network interface.
- The task of writing a kernel driver is complex. There are some references to help.
- To outline options available using a bare bones model of a character device driver.

Designing a character driver interface

- The main character driver interface is based on a stream of bytes.
- However, devices don't fit this description: controller for robotic arm needs to move and rotate each joint ...
- There are other ways to communicate with device drivers than just read and write:

ioctl(): allows passing two arguments to a driver, and these can have any meaning - by convention.

The first argument is a command to select functions in a driver, and the second is a ptr to a structure as a container for the input and output parameters.

It allows any programming interface and is common when a driver and application are closely linked. Yet, ioctl is deprecated in kernel, and it is hard to find drivers with new uses of ioctl accepted upstream. kernel maintainers dislike ioctl because kernel code and application code become too interdependent, and it is hard to keep both of them in step across kernel versions and architectures.

sysfs: preferred way to do things now, as in GPIO interface. The advantages are that it is self-documenting, so long there are descriptive names for the files. It is also scriptable because the file's content is usually text strings. On the other hand, the requirement for each file to contain a single value makes it hard to achieve atomicity if you need to change more than one value at a time. Conversely, ioctl passes all its arguments in a structure, in a single function call.

mmap(): direct access to kernel buffers and hardware registers by mapping kernel memory into user space, thus bypassing the kernel. It will still need kernel code to handle interrupts and DMA. a subsystem that encapsulates this idea is known as uio, which is short for user I/O. more documentation is in Documentation/DocBook/uiohowto, with example drivers in drivers/uio.

sigio: send a signal from a driver using the kernel function named kill_fasync() to notify applications of an event, such as input becoming ready or an interrupt being received. By convention, SIGIO is used, but it could be any. There are some examples in the UIO driver, drivers/uio/uio.c, and in the RTC driver, drivers/char/rtc.c. remains little-used.

debugfs: another pseudo filesystem that represents kernel data as files and directories, similar to proc and sysfs, with the main distinction that debugfs must not contain information that is needed for the normal operation of the system; it is for debug and trace information only. It is mounted as mount -t debugfs debug /sys/kernel/debug. More in Documentation/filesystems/debugfs.txt kernel documentation.

proc: deprecated for all new code unless it relates to processes. However, use proc to publish any information. And, unlike sysfs and debugfs, it is available to non-GPL modules.

netlink: a socket protocol family. AF_NETLINK creates a socket that links the kernel space to the user space. It was originally created so that network tools could communicate with the Linux network code to access the routing tables and other details. It is also used by udev to pass events from the kernel to the udev daemon, and is very rarely used in general device drivers.

- There are many examples of each of the preceding approaches in the kernel source code.
- The only universal rule is the principle of least astonishment. In other words, application writers who are using a driver should find that everything works in a logical way, without any quirks or oddities.

The anatomy of a device driver

- looking at the code for a simple device driver.

```
// Here is the start of a device driver named dummy,
// which creates four devices that can be accessed through /dev/dummy0 to /dev/dummy3:
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/init.h>
#include <linux/fs.h>
#include <linux/device.h>
#define DEVICE_NAME "dummy"
#define MAJOR_NUM 42
#define NUM_DEVICES 4
static struct class *dummy_class;
// Next, define the dummy_open(), dummy_release(), dummy_read(), and dummy_write()
// functions for the character device interface:
static int dummy_open(struct inode *inode, struct file *file)
{
    pr_info("%s\n", __func__);
    return 0;
}
static int dummy_release(struct inode *inode, struct file *file)
{
    pr_info("%s\n", __func__);
    return 0;
}
static ssize_t dummy_read(struct file *file, char *buffer, size_t length, loff_t * offset)
{
    pr_info("%s %u\n", __func__, length);
    return 0;
}
```

```

static ssize_t dummy_write(struct file *file, const char *buffer, size_t length, loff_t * offset)
{
    pr_info("%s %u\n", __func__, length);
    return length;
}
// After that, initialize a file_operations structure and define the dummy_init() and dummy_exit() functions,
// which are called when the driver is loaded and unloaded:
struct file_operations dummy_fops = {
    .owner = THIS_MODULE,
    .open = dummy_open,
    .release = dummy_release,
    .read = dummy_read,
    .write = dummy_write,
};
int __init dummy_init(void)
{
    int ret;
    int i;
    printk("Dummy loaded\n");
    ret = register_chrdev(MAJOR_NUM, DEVICE_NAME, &dummy_fops);
    if (ret != 0) return ret;
    dummy_class = class_create(THIS_MODULE,
    DEVICE_NAME);
    for (i = 0; i < NUM_DEVICES; i++) {
        device_create(dummy_class, NULL, MKDEV(MAJOR_NUM, i), NULL, "dummy%d", i);
    }
    return 0;
}
void __exit dummy_exit(void)
{
    int i;
    for (i = 0; i < NUM_DEVICES; i++) {
        device_destroy(dummy_class, MKDEV(MAJOR_NUM, i));
    }
    class_destroy(dummy_class);
    unregister_chrdev(MAJOR_NUM, DEVICE_NAME);
    printk("Dummy unloaded\n");
}
// At the end of the code, module_init and module_exit are macros that specify the functions to be called
// when the module is loaded and unloaded:
module_init(dummy_init);
module_exit(dummy_exit);
// The closing three macros named MODULE_* add some basic information about the module:
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Me");
MODULE_DESCRIPTION("A dummy driver");

```

- This information can be retrieved from the compiled kernel module using the **modinfo** command.
- When the module is loaded, the dummy_init() function is called.
- The point at which it becomes a character device is when it makes the call to register_chrdev, passing a ptr to struct file_operations, which contains pointers to the four functions that the driver implements.

- While `register_chrdev` tells the kernel that there is a driver with a major number of 42, it doesn't say anything about the class of driver, and so it will not create an entry in `/sys/class`.
- Without an entry in `/sys/class`, the device manager cannot create device nodes.
- So, the next few lines of code create a device class, `dummy`, and four devices of that class called `dummy0` to `dummy3`.
- The result is that the `/sys/class/dummy` directory is created when the driver is initialized, containing subdirectories `dummy0` to `dummy3`.
- Each of the subdirectories contains a file, `dev`, that contains the major and minor numbers of the device.
- This is all that a device manager needs to create device nodes: `/dev/dummy0` to `/dev/dummy3`.
- The `dummy_exit()` function has to release the resources claimed by `dummy_init()`, which means freeing up the device class and major number.
- The file operations for this driver are implemented by `dummy_open()`, `dummy_read()`, `dummy_write()`, and `dummy_release()`, and they are called when a user space program calls `open(2)`, `read(2)`, `write(2)`, and `close(2)`, respectively.
- They just print a kernel message to see that they were called.

Demonstrate this from the command line using the echo command:

```
# echo hello > /dev/dummy0
dummy_open
dummy_write 6
dummy_release
```

- kernel messages are printed to the console by default.
- If not logged onto the console, see the kernel messages by using the `dmesg` command.
- The full source code for this driver is less than 100 lines, and it illustrates how the linkage between a device node and driver code works; how the device class is created, allowing a device manager to create device nodes automatically when the driver is loaded; and how the data is moved between the user and kernel spaces.

Kernel

Compiling kernel modules

to compile and test a module on a target system.

- **copy** it into the kernel source tree and modify makefiles to build it, or
- **compile** it as a module out of tree.

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Compiling kernel modules - to compile and test a module on a target system.

- copy it into the kernel source tree and modify makefiles to build it, or compile it as a module out of tree.
- To build it, out of tree, it needs a simple makefile that uses the kernel build system to do all the hard work:
 `LINUXDIR := $(HOME)/el/build/linux`
 `obj-m := dummy.o ## invokes kernel build rule to take source file dummy.c, and create a kernel module`
 `all:`
 `make ARCH=arm CROSS_COMPILE=arm-cortex_a8-linuxgnueabihf- \`
 `-C $(LINUXDIR) M=$(shell pwd)`
 `clean:`
 `make -C $(LINUXDIR) M=$(shell pwd) clean`
- Set LINUXDIR to directory of the kernel for the target device that the module will run on.
- Note: Kernel modules are not binary compatible between kernel releases and configurations: the module will only load on the kernel it was compiled with.
- To build a driver in the kernel source tree, the procedure is quite simple.
 Choose a directory appropriate to the type of driver needed . The driver is a basic character device, so put `dummy.c` in `drivers/char`. Then, edit the makefile in the directory, and add a line to build the driver unconditionally as a module, as follows:
- `obj-m += dummy.o`
- Alternatively, you can add the following line to build it unconditionally as a builtin:
- `obj-y += dummy.o`
- To make the driver optional, add an menu option to the Kconfig file and make compilation conditional on the configuration option,

Kernel

Loading kernel modules

Loading kernel modules

- To load, unload, and list modules use the simple insmod, lsmod, and rmmod commands, respectively.

- e.g.:

```
# insmod /lib/modules/4.8.12-yoctostandard/  
kernel/drivers/dummy.ko
```

```
# lsmod
```

```
Tainted: G
```

```
dummy 2062 0 - Live 0xbf004000 (O)
```

```
# rmmod dummy
```

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Loading kernel modules

- load, unload, and list modules using the simple insmod, lsmod, and rmmod commands, respectively.
- e.g.

```
# insmod /lib/modules/4.8.12-yoctostandard/  
kernel/drivers/dummy.ko  
# lsmod  
Tainted: G  
dummy 2062 0 - Live 0xbf004000 (O)  
# rmmod dummy
```
- or place module under /lib/modules/<kernel release>, and create a module dependency using depmod -a

```
# depmod -a  
# ls /lib/modules/4.8.12-yocto-standard  
kernel modules.alias modules.dep modules.symbols
```
- information in the modules.* files is used by the modprobe command to locate a module by name rather than its full path.
- modprobe has many other features, described in the modprobe(8) manual page.
- to get a module to talk to some real piece of hardware - bind the driver to that hardware either by way of the device tree or platform data.

Kernel

Discovering the hardware configuration

- The dummy driver lacks interaction with real hardware since it only manipulates memory structures.
- Device drivers are usually written to interact with hardware.
- Part of that is being able to discover the hardware in the first place, bearing in mind that it may be at different addresses in different configurations.
- In some cases, the hardware provides the information itself.

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Discovering the hardware configuration - Device drivers are usually written to interact with hardware.

- dummy driver lacks interaction with real hardware since it only manipulates memory structures. Mainly, being able to discover hardware since it may be at different addresses in different configurations.
- Sometimes, the hardware provides the information itself.
On a discoverable bus .. PCI or USB - using a query mode to return resource requirements and unique id. The kernel matches identifier and other characteristics with the device drivers and connects them.
- However, most hardware blocks on embedded boards do not have such identifiers.
- Information is needed in the form of a device tree or as C structures known as platform data.
- In the standard driver model for Linux, device drivers register themselves with the appropriate subsystem: PCI, USB, open firmware (device tree), platform device, and so on.
- Registration includes an identifier and a callback function, called a probe function, called if there is a match between the ID of the hardware and the ID of the driver.
- For PCI and USB, the ID is based on the vendor and the product IDs of the devices; for device trees and platform devices, it is a name (a text string).

Device trees - to show how the Linux device drivers hook up with this information.

using Arm Versatile board, arch/arm/boot/dts/versatile-ab.dts, for which the Ethernet adapter is defined:

```
net@10010000 {  
    compatible = "smc,lan91c111";  
    reg = <0x10010000 0x10000>;  
    interrupts = <25>;  
};
```

Pay attention to the compatible property of this node.

This string value will reappear later in the source code for the Ethernet adapter.

The platform data

- In the absence of device tree support, there is a fallback method of describing hardware using C structures, known as the platform data.
- Each piece of hardware is described by struct platform_device, which has a name and a pointer to an array of resources.
- The resource's type is determined by flags, which include the following:
 - IORESOURCE_MEM: This is the physical address of a region of memory.
 - IORESOURCE_IO: This is the physical address or port number of I/O registers.
 - IORESOURCE_IRQ: This is the interrupt number.
- an example of the platform data for an Ethernet controller taken from arch/arm/mach-versatile/core.c, which has been edited for clarity:

```
#define VERSATILE_ETH_BASE 0x10010000
#define IRQ_ETH 25
static struct resource smc91x_resources[] = {
    [0] = {
        .start = VERSATILE_ETH_BASE,
        .end = VERSATILE_ETH_BASE + SZ_64K - 1,
        .flags = IORESOURCE_MEM,
    },
    [1] = {
        .start = IRQ_ETH,
        .end = IRQ_ETH,
        .flags = IORESOURCE_IRQ,
    },
};
static struct platform_device smc91x_device = {
    .name = "smc91x",
    .id = 0,
    .num_resources = ARRAY_SIZE(smc91x_resources),
    .resource = smc91x_resources,
};
```

It has a memory area of 64 KB and an interrupt.

The platform data has to be registered with the kernel, usually when the board is initialized:

```
void __init versatile_init(void)
{
    platform_device_register(&versatile_flash_device);
    platform_device_register(&versatile_i2c_device);
    platform_device_register(&smc91x_device);
    [...]
}
```

- The platform data shown here is functionally equivalent to the previous device tree source, except for the name field, which takes the place of the compatible property.

Linking hardware with device drivers

In the preceding section, an Ethernet adapter is described using a device tree and using platform data.

The corresponding driver code is in drivers/net/ethernet/smc/smc91x.c, and it works with both the device tree and platform data.

Here is the initialization code, edited for clarity:

```
static const struct of_device_id smc91x_match[] = {
    { .compatible = "smc,lan91c94", },
    { .compatible = "smc,lan91c111", },
    {},
};
MODULE_DEVICE_TABLE(of, smc91x_match);
static struct platform_driver smc_driver = {
    .probe = smc_drv_probe,
    .remove = smc_drv_remove,
    .driver = {
        .name = "smc91x",
        .of_match_table = of_match_ptr(smc91x_match),
    },
};
static int __init smc_driver_init(void)
{
    return platform_driver_register(&smc_driver);
}
static void __exit smc_driver_exit(void)
{
    platform_driver_unregister(&smc_driver);
}
module_init(smc_driver_init);
module_exit(smc_driver_exit);
```

- When the driver is initialized, it calls `platform_driver_register()`, pointing to `struct platform_driver`, in which there is a callback to a probe function, a driver name, `smc91x`, and a pointer to `struct of_device_id`.
- If this driver has been configured by the device tree, the kernel will look for a match between the compatible property in the device tree node and the string being pointed to by the compatible structure element.
- For each match, it calls the probe function.

On the other hand, if it was configured through platform data, the probe function will be called for each match on the string pointed to by `driver.name`.

- The probe function extracts information about the interface:

```
static int smc_drv_probe(struct platform_device *pdev)
{
    struct smc91x_platdata *pd =
        dev_get_platdata(&pdev->dev);
    const struct of_device_id *match = NULL;
    struct resource *res, *ires;
    int irq;
    res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
    ires = platform_get_resource(pdev, IORESOURCE_IRQ, 0);
    [...]
    addr = ioremap(res->start, SMC_IO_EXTENT);
    irq = ires->start;
    [...]
}
```

- The calls to `platform_get_resource()` extract the memory and irq information from either the device tree or the platform data.

It is up to the driver to map the memory and install the interrupt handler.

- The third parameter, which is zero in both of the previous cases, comes into play if there is more than one resource of that particular type.
- Device trees configuration of more than just basic memory ranges and interrupts.
- There is a section of code in the probe function that extracts optional parameters from the device tree.
- In this snippet, it gets the register io-width property:


```
match = of_match_device(of_match_ptr(smc91x_match), &pdev->dev);
if (match) {
    struct device_node *np = pdev->dev.of_node;
    u32 val;
    [...]
    of_property_read_u32(np, "reg-io-width", &val);
    [...]
}
```
- For most drivers, specific bindings are documented in Documentation/devicetree/bindings.
- For this particular driver, the information is in Documentation/devicetree/bindings/net/smc911x.txt.
- The main thing to remember here is that drivers should register a probe function and enough information for the kernel to call probe, as it finds matches with the hardware it knows about.
- The linkage between the hardware described by the device tree and the device driver is done through the compatible property.
- The linkage between platform data and a driver is done through the name.

Summary Notes

- Device Drivers
- Kernel Modules.

Further reading

- Linux Kernel Development, 3rd Edition, by Robert Love
- Linux Weekly News: <https://lwn.net/Kernel>
- Async IO on Linux: select, poll, and epoll, by Julia Evans:
<https://jvns.ca/blog/2017/06/03/async-io-on-linux--select--poll--and-epoll>
- Essential Linux Device Drivers, 1st Edition, by Sreekrishnan Venkateswaran

1. Device drivers have the job of handling devices, usually physical hardware but sometimes virtual interfaces, and presenting them to the user space in a consistent and useful way.
2. Linux device drivers fall into three broad categories: character, block, and network. Of the three, the character driver interface is the most flexible and therefore, the most common.
3. Linux drivers fit into a framework known as the driver model, which is exposed through sysfs. The entire state of the devices and drivers is visible in /sys.
4. Each embedded system has its own unique set of hardware interfaces and requirements.
5. Linux provides drivers for most standard interfaces, and by selecting the right kernel configuration, it gets a working target board very quickly.
6. For non-standard components, it will need to add own device support. In some cases, the issue is side-stepped by using generic drivers for GPIO, I2C, and so on, and then write user space code to do the work. This is a good starting point, as it provides the chance to become familiar with the hardware without writing kernel code.
7. Writing kernel drivers is not particularly difficult, but there is a need to code carefully so as not to compromise the stability of the system.

Quiz

Canvas

- 10 Questions
- 45 Mins
- 6 Attempts
- Highest Retained

2021

Embedded Linux Design and Programming

Raghav Vinjamuri

6-43

Quiz

Good Luck!

Assignment

1. Create a simple char driver.
2. Show the sysfs entry created for it.

Assignment

1. Create a simple char driver.
2. Show the sysfs entry created for it.

Summary

In This Chapter

- What does the kernel do?
- Details of Linux Architecture.
- Recap - Device Tree, bootloader
- Module Programming, Device Drivers
- Assignment
- Quiz

CHAPTER OVERVIEW

In This Chapter, We Discussed

1. What does the kernel do?
2. Details of Linux Architecture.
3. Recap - Device Tree, bootloader
4. Module Programming, Device Drivers
5. Assignment
6. Quiz

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Review & Questions

- Review
- Q & A

2021

Embedded Linux Design and Programming

Raghav Vinjamuri

6-46

Q&A